Department of Electrical Engineering

Indian Institute of Technology, Kharagpur

Algorithms, AI and ML Laboratory (EE22202)

Spring, 2025-26

Report 09: Fashion MNIST Classification using Convolutional Neural Networks

Name: Arijit Dey

Roll No: 24IE10001

Fashion MNIST Classification using Convolutional Neural Networks

Data Loading and Preprocessing

Here we load the FashionMNIST dataset, and prepare the data loaders for training, validation, and testing. A 20% split of the training data is used for validation. The device is set to 'cuda' if available, otherwise 'cpu'.

Data Transformation and Loading

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
train_dataset = datasets.FashionMNIST("F_MNIST_data", download = True, train = True, transform = transform )
test_dataset = datasets.FashionMNIST("F_MNIST_data", download = True, train = False, transform = transform )

indices = list(range(len(train_dataset)))
np.random.shuffle(indices)
split = int(0.2 * len(train_dataset))
val_ids , train_ids = indices[:split], indices[split:]
```

DataLoader Setup

```
train_sampler = torch.utils.data.SubsetRandomSampler(train_ids)
val_sampler = torch.utils.data.SubsetRandomSampler(val_ids)

BATCH_SIZE = 128
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size = BATCH_SIZE, sampler = train_sampler)
val_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size = BATCH_SIZE, sampler = val_sampler)
test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size = BATCH_SIZE, shuffle = False)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Data Inspection

```
print("Length of train data is " + str(len(train_sampler)))
print("Length of test data is " + str(len(test_dataset)))
print("Length of validation data is " + str(len(val_sampler)))

image, label = next(iter(train_loader))
print(image[0].shape, label.shape)

desc = ["T-shirt/top ", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle Boot"]
print(desc[label[0].item()])
plt.imshow(image[0].numpy().squeeze(), cmap="gray")
```

```
Result
Length of train data is 48000
Length of test data is 10000
Length of validation data is 12000
torch.Size([1, 28, 28]) torch.Size([128])
Sandal
```

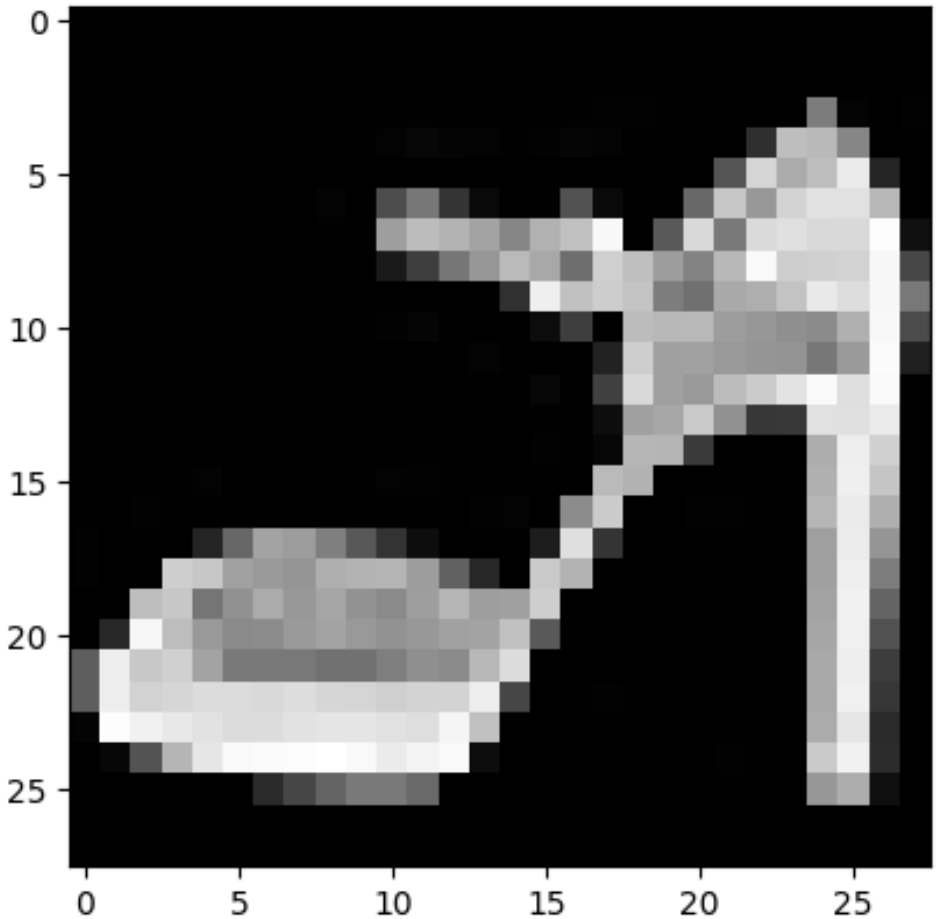

Figure 1: Sample Image from FashionMNIST Dataset

Model Definition: `BaseCNN`

The `BaseCNN` class models our entire classification model pipeline. We inherit it from PyTorch's `torch.nn.Module` class to get access to PyTorch's pre-built layers like `Conv2D`, `BatchNorm2D`, `ReLU` and `Sequential`. The class allows for customisation of kernel size, stride, number of convolutional blocks, number of filters, fully connected layer hidden size, dropout probability, and batch normalization.

BaseCNN Class Definition

```
class BaseCNN(nn.Module):
    def __init__(self,
                 k=2,
                 s=1,
                 num_blocks=3,
                 filters=64,
                 fc_hidden=64,
```

```
                 dropout_p=0.0,
                 batch_norm=False):
        super(BaseCNN, self).__init__()
        self.num_blocks = num_blocks
        self.conv_layers = nn.ModuleList()

        in_channels = 1
        current_size = 28

        for i in range(num_blocks):
            # Use padding to try and keep spatial dimensions if s=1
            padding = (k - 1) // 2 if s == 1 else (k // 2)
            conv = nn.Conv2d(in_channels, filters, kernel_size=k, stride=s, padding=padding)
            self.conv_layers.append(conv)

            if batch_norm:
                self.conv_layers.append(nn.BatchNorm2d(filters))

            self.conv_layers.append(nn.ReLU())

            current_size = (current_size + 2 * padding - k) // s + 1

            if current_size > 1:
                self.conv_layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
                current_size = current_size // 2

            in_channels = filters

        self.flat_size = filters * current_size * current_size

        self.fc = nn.Sequential(
            nn.Linear(self.flat_size, fc_hidden),
            nn.ReLU(),
            nn.Dropout(dropout_p) if dropout_p > 0 else nn.Identity(),
            nn.Linear(fc_hidden, 10)
        )
```

The `forward` function

The `forward` method passes the input `x` through the defined convolutional and fully connected layers. It can also optionally return intermediate activations from the convolutional layers, which is useful for visualisation and debugging.

```
    def forward(self, x, return_activations=False):
        activations = []
        for layer in self.conv_layers:
            x = layer(x)
            if isinstance(layer, nn.Conv2d):
                if return_activations:
                    activations.append(x)

```

```
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        if return_activations:
            return x, activations
        return x
```

Training and Reporting Function

This section defines the `train_and_report` function, which handles the training loop, validation, and testing of a given CNN model. It calculates and prints training, validation, and test accuracies, and visualises loss history and convolutional layer activations at certain epochs.

train_and_report Function Definition

```
def train_and_report(model, num_epochs=50, lr=0.001):
    # ... SETUP CODE OMITTED

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * images.size(0)

        train_loss = running_loss / len(train_loader.sampler)
        train_losses.append(train_loss)

        model.eval()
        v_loss = 0.0
        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)
                v_loss += loss.item() * images.size(0)
        val_loss = v_loss / len(val_loader.sampler)
        val_losses.append(val_loss)

    # PROGRESS REPORTING AND PLOTTING CODE OMITTED...
```

Experiment 1: Baseline

This experiment establishes a baseline performance for the CNN model using default parameters: kernel size of 2, stride of 1, 3 convolutional blocks, 64 filters, 64 fully connected hidden units, no dropout, and no batch normalisation.

Baseline Model Training

```
model_baseline = BaseCNN(k=2, s=1, num_blocks=3, filters=64, fc_hidden=64, dropout_p=0.0,
batch_norm=False)
baseline_acc = train_and_report(model_baseline, num_epochs=50)
```

Result

```
Experiment 1: Baseline
Total Trainable Parameters: 50314
Epoch 1/50, Train Loss: 0.7333, Val Loss: 0.5294
Epoch 10/50, Train Loss: 0.2646, Val Loss: 0.3054
Epoch 20/50, Train Loss: 0.1827, Val Loss: 0.3110
Epoch 30/50, Train Loss: 0.1256, Val Loss: 0.3463
Epoch 40/50, Train Loss: 0.0884, Val Loss: 0.4516
Epoch 50/50, Train Loss: 0.0622, Val Loss: 0.5607
Final Accuracies: Train: 97.44%, Val: 88.62%, Test: 88.24%
```

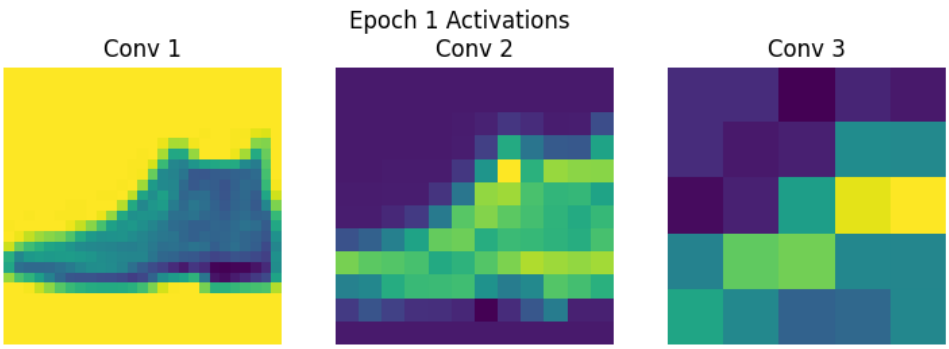

Figure 2: Epoch 1 Activations

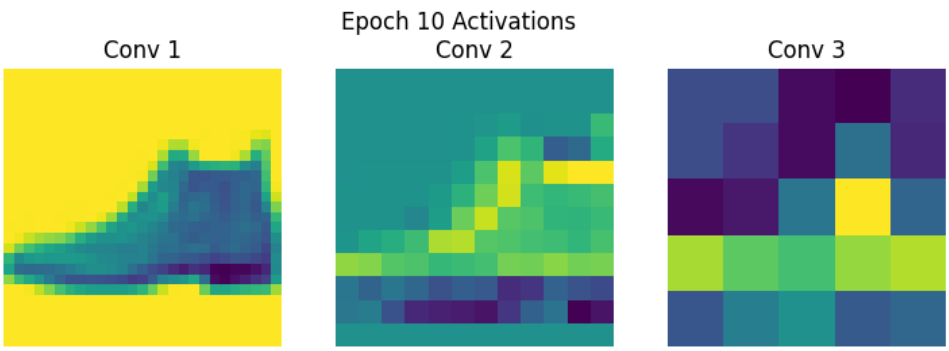

Figure 3: Epoch 10 Activations

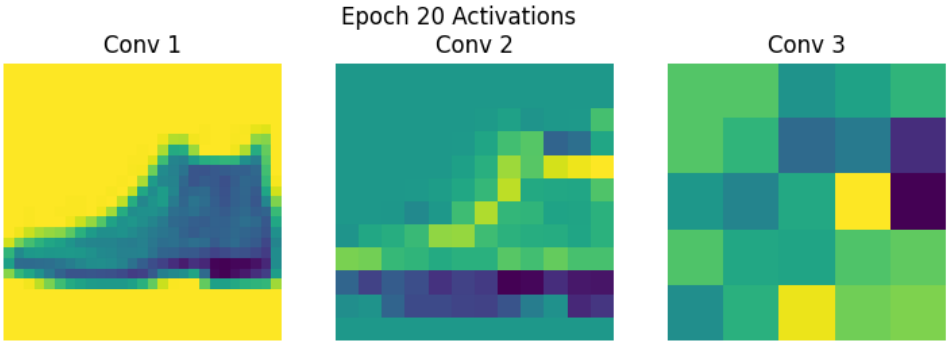

Figure 4: Epoch 20 Activations

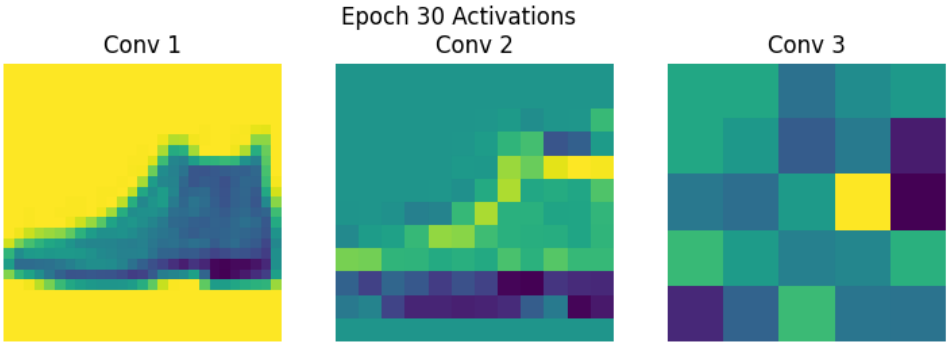

Figure 5: Epoch 30 Activations

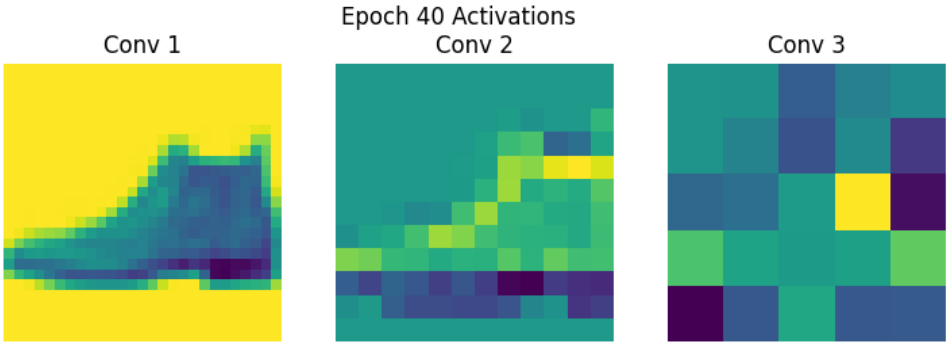

Figure 6: Epoch 40 Activations

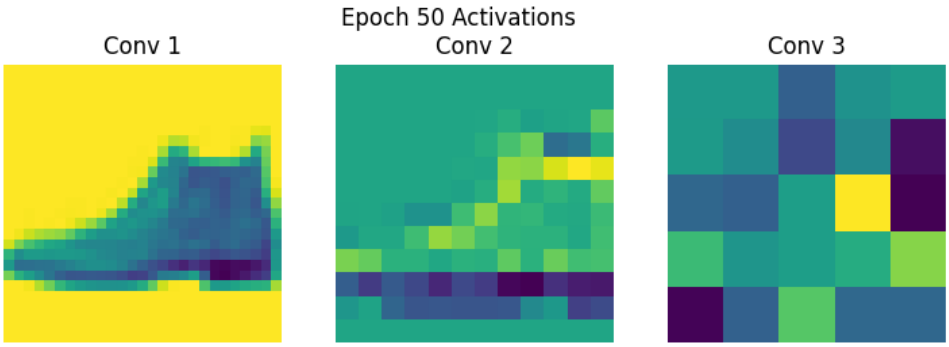

Figure 7: Epoch 50 Activations

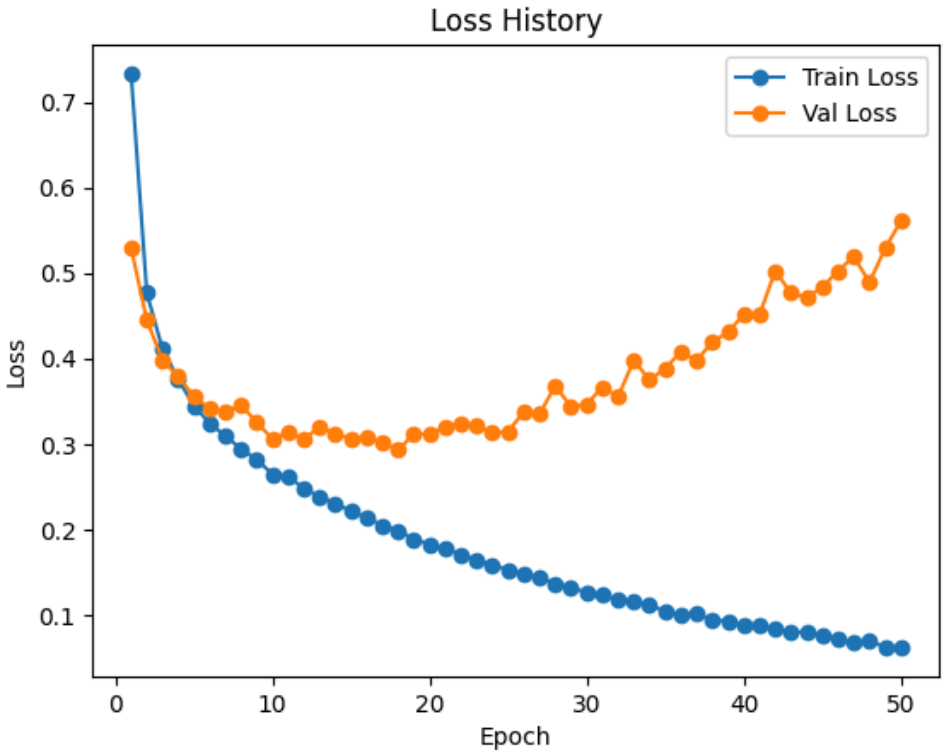

Figure 8: Loss History

Observations

The baseline model achieves a test accuracy of 88.24%. The training loss consistently decreases, but the validation loss starts to increase after a certain point, indicating potential overfitting. The activation maps show features being learned by the convolutional layers over epochs.

Experiment 2: Kernel Size and Stride Ablation

This experiment investigates the effect of using a larger kernel size and stride on the CNN model's performance. The model is configured with a kernel size of 5 and a stride of 2, while keeping other parameters similar to the baseline.

Kernel Size and Stride Ablation Model Training

```
model_ks = BaseCNN(k=5, s=2, num_blocks=3, filters=64, fc_hidden=64, dropout_p=0.0,
batch_norm=False)
ks_acc = train_and_report(model_ks, num_epochs=50)
```

Result

```
Total Trainable Parameters: 211402
Epoch 1/50, Train Loss: 0.6746, Val Loss: 0.4608
Epoch 10/50, Train Loss: 0.1843, Val Loss: 0.2525
Epoch 20/50, Train Loss: 0.0857, Val Loss: 0.3128
Epoch 30/50, Train Loss: 0.0398, Val Loss: 0.4987
Epoch 40/50, Train Loss: 0.0313, Val Loss: 0.5306
Epoch 50/50, Train Loss: 0.0272, Val Loss: 0.6256
Final Accuracies: Train: 99.45%, Val: 90.63%, Test: 90.19%
```

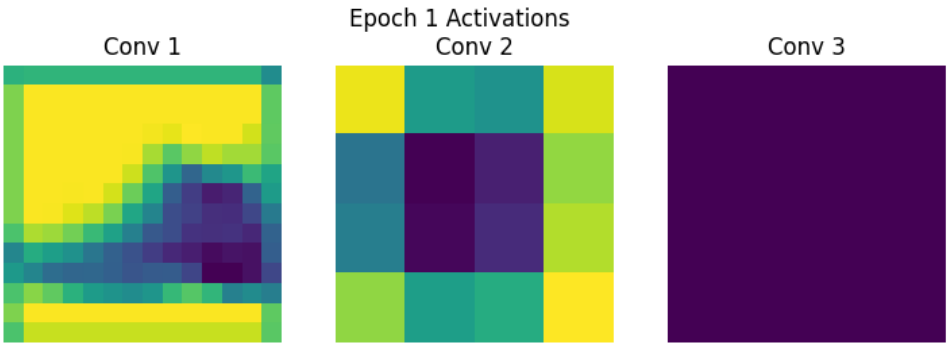

Figure 9: Epoch 1 Activations (KS/Stride Model)

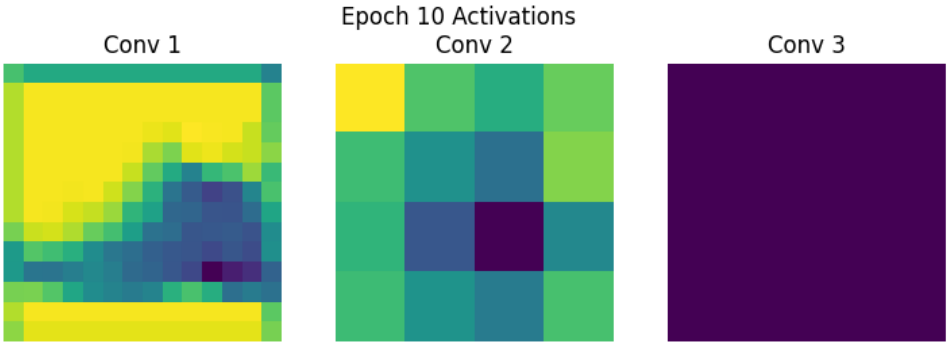

Figure 10: Epoch 10 Activations (KS/Stride Model)

Figure 11: Epoch 20 Activations (KS/Stride Model)

Figure 12: Epoch 30 Activations (KS/Stride Model)

Figure 13: Epoch 40 Activations (KS/Stride Model)

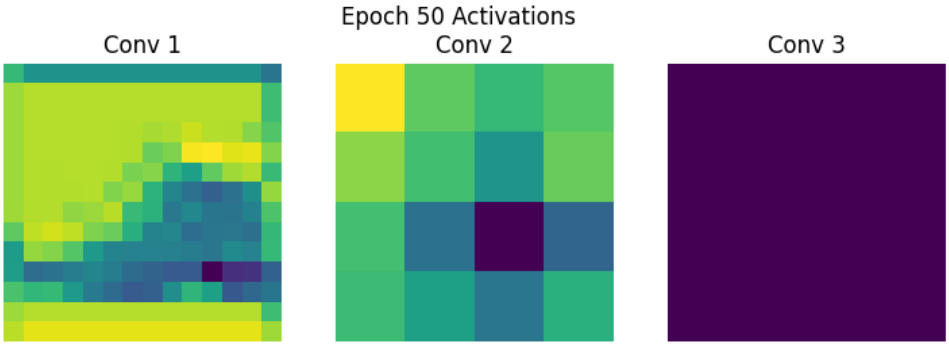

Figure 14: Epoch 50 Activations (KS/Stride Model)

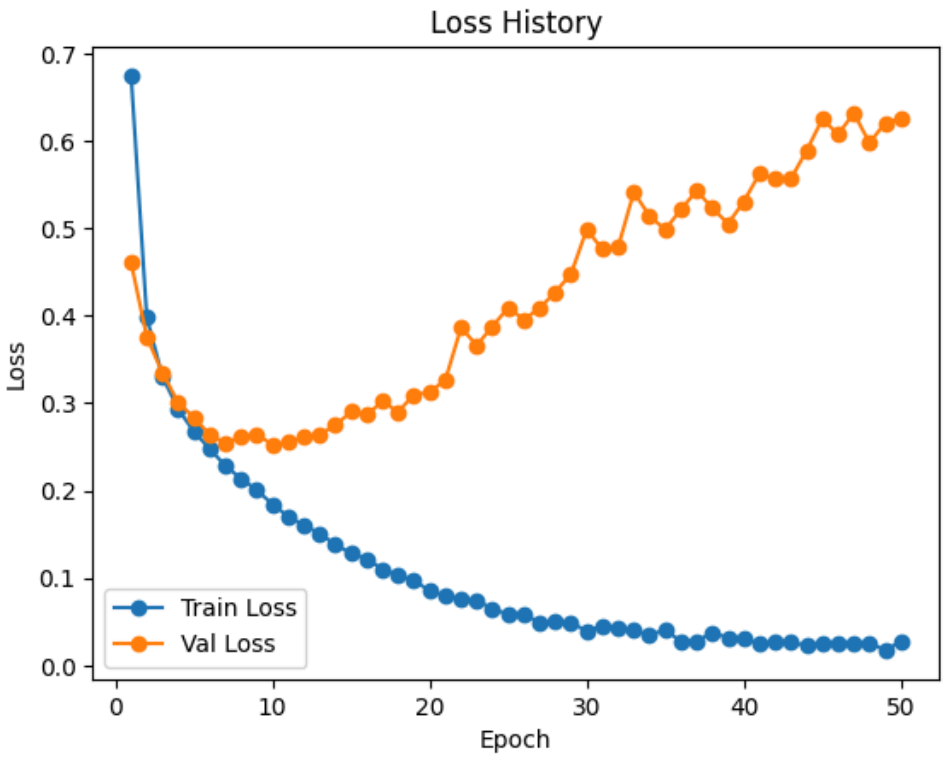

Figure 15: Loss History (KS/Stride Model)

Observations

This model achieved a test accuracy of 90.19%, which is an improvement over the baseline. The training loss decreases significantly, but the validation loss shows a clear upward trend

after around epoch 10, indicating severe overfitting. The higher number of trainable parameters (211402) compared to the baseline (50314) suggests that increasing the kernel size and stride without regularisation can lead to a more complex model that overfits the training data.

Experiment 3: Depth Ablation (Shallower)

This experiment examines the impact of reducing the network's depth on its performance. The model is configured with only 2 convolutional blocks, making it shallower than the baseline (3 blocks).

Shallower Model Training

```
model_shallow = BaseCNN(k=2, s=1, num_blocks=2, filters=64, fc_hidden=64, dropout_p=0.0,
batch_norm=False)
shallow_acc = train_and_report(model_shallow, num_epochs=50)
```

Result

```
Experiment 3: Depth Ablation (shallower)
Total Trainable Parameters: 164938
Epoch 1/50, Train Loss: 0.5815, Val Loss: 0.4054
Epoch 10/50, Train Loss: 0.1881, Val Loss: 0.2524
Epoch 20/50, Train Loss: 0.0924, Val Loss: 0.3127
Epoch 30/50, Train Loss: 0.0410, Val Loss: 0.4491
Epoch 40/50, Train Loss: 0.0256, Val Loss: 0.6016
Epoch 50/50, Train Loss: 0.0202, Val Loss: 0.6897
Final Accuracies: Train: 99.31%, Val: 90.11%, Test: 90.24%
```

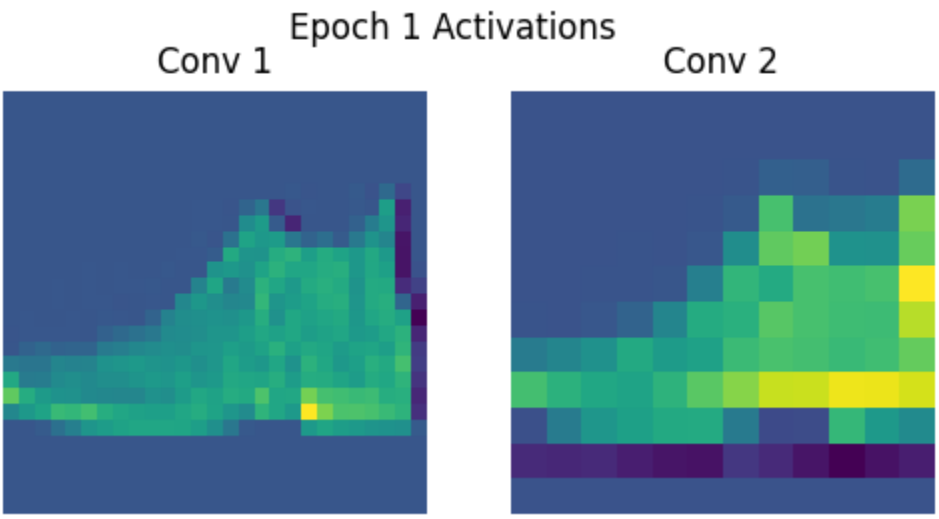

Figure 16: Epoch 1 Activations (Shallower Model)

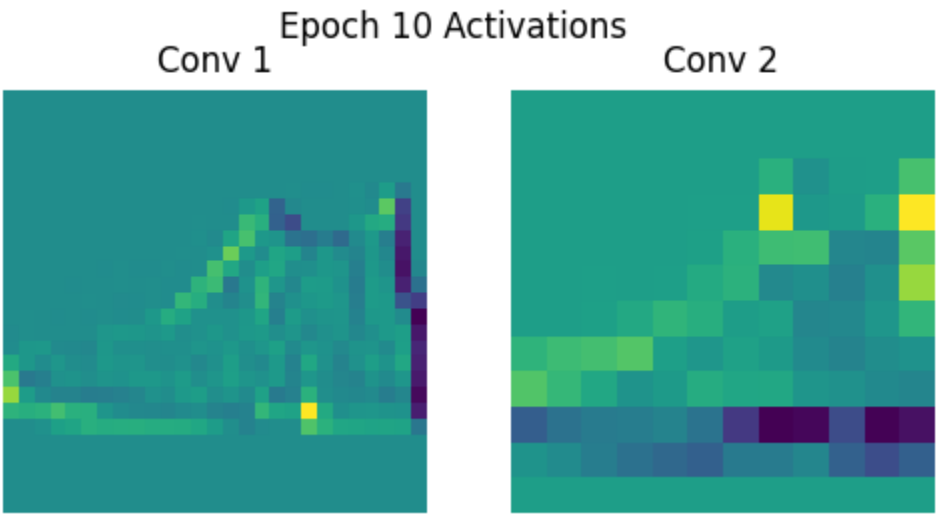

Figure 17: Epoch 10 Activations (Shallower Model)

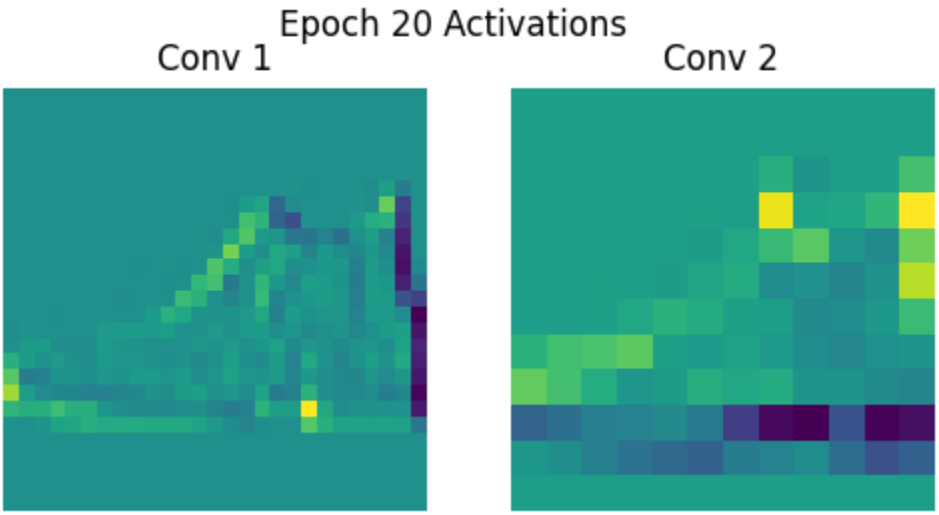

Figure 18: Epoch 20 Activations (Shallower Model)

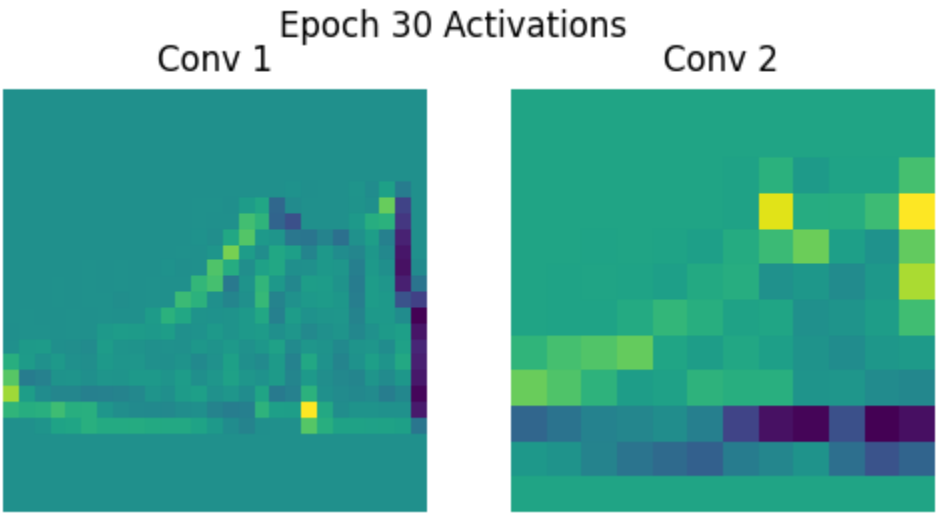

Figure 19: Epoch 30 Activations (Shallower Model)

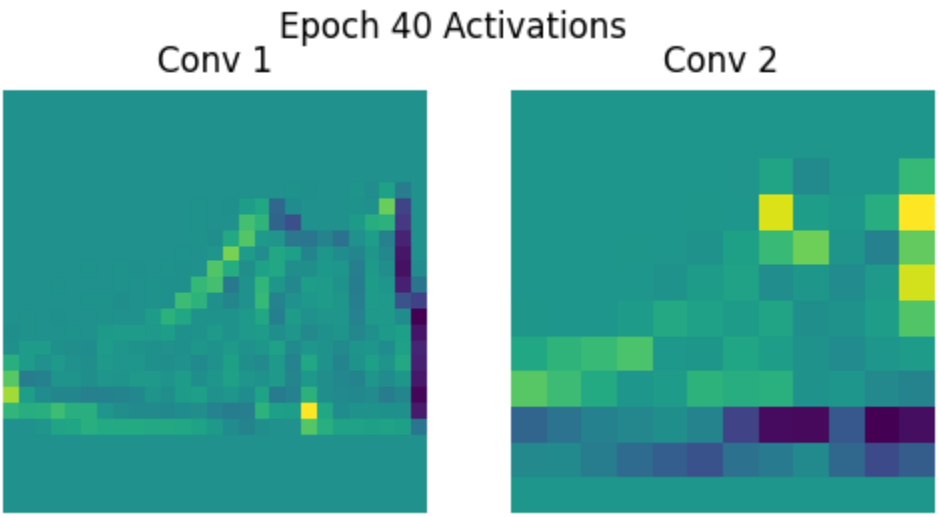

Figure 20: Epoch 40 Activations (Shallower Model)

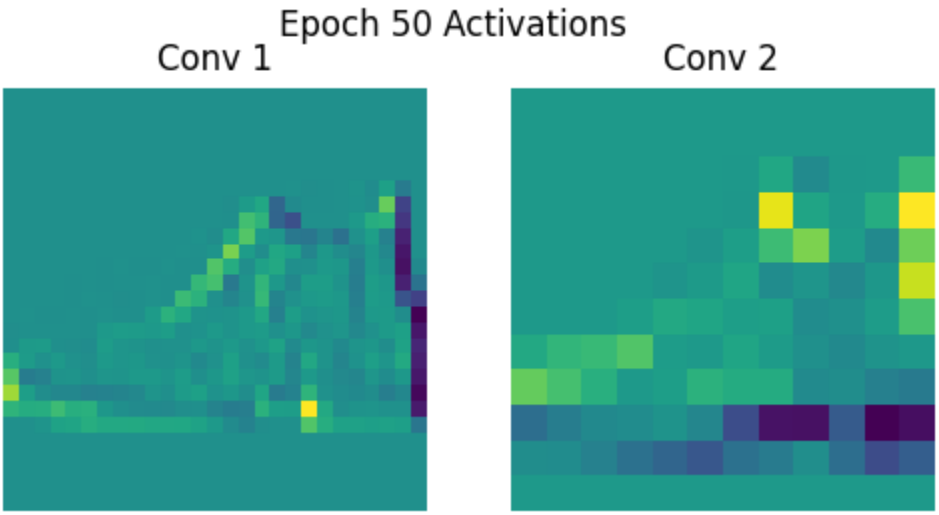

Figure 21: Epoch 50 Activations (Shallower Model)

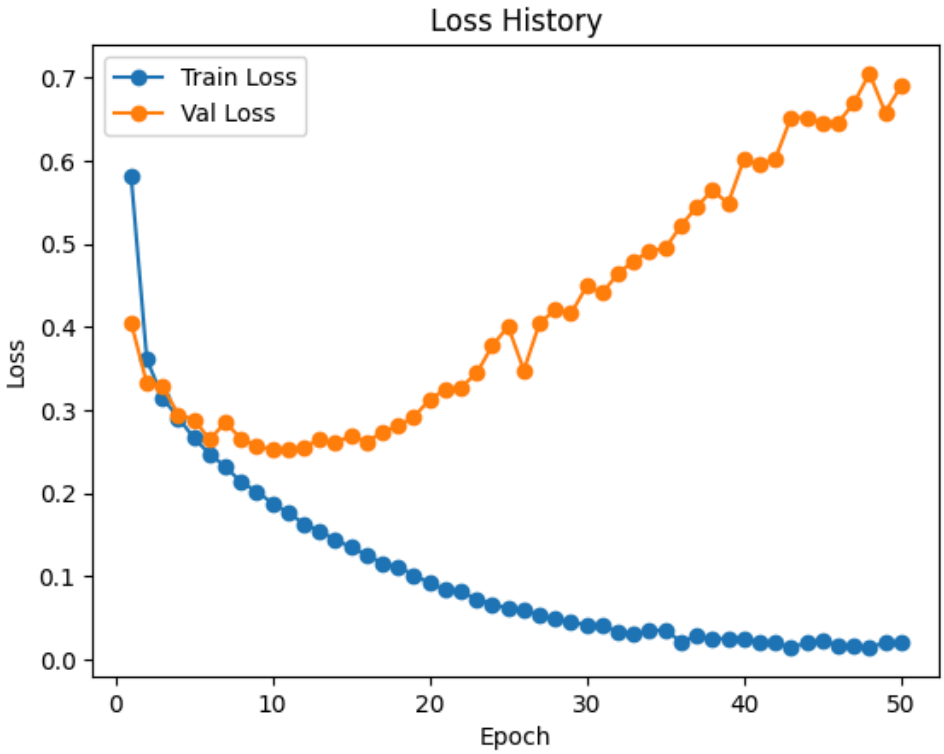

Figure 22: Loss History (Shallower Model)

Observations

The shallower model achieves a test accuracy of 90.24%, which is slightly better than the baseline and similar to the KS/Stride model. However, it also shows signs of overfitting, with validation loss increasing significantly in later epochs. This suggests that even with fewer layers, the model can still be complex enough to overfit if not properly regularised.

Experiment 4: Dropout (p = 0.25)

This experiment evaluates the effect of adding dropout regularization to the CNN model to mitigate overfitting. The model is configured with a dropout probability of 0.25, while maintaining the baseline's architecture.

Dropout Model Training

```
model_dropout = BaseCNN(k=2, s=1, num_blocks=3, filters=64, fc_hidden=64, dropout_p=0.25,
batch_norm=False)
dropout_acc = train_and_report(model_dropout, num_epochs=50)
```

Result

```
Experiment 4: Dropout (p = 0.25)
Total Trainable Parameters: 50314
Epoch 1/50, Train Loss: 0.8317, Val Loss: 0.5271
Epoch 10/50, Train Loss: 0.3154, Val Loss: 0.3161
Epoch 20/50, Train Loss: 0.2469, Val Loss: 0.2894
Epoch 30/50, Train Loss: 0.2022, Val Loss: 0.3064
Epoch 40/50, Train Loss: 0.1711, Val Loss: 0.3313
Epoch 50/50, Train Loss: 0.1520, Val Loss: 0.3624
Final Accuracies: Train: 95.32%, Val: 89.01%, Test: 89.19%
```

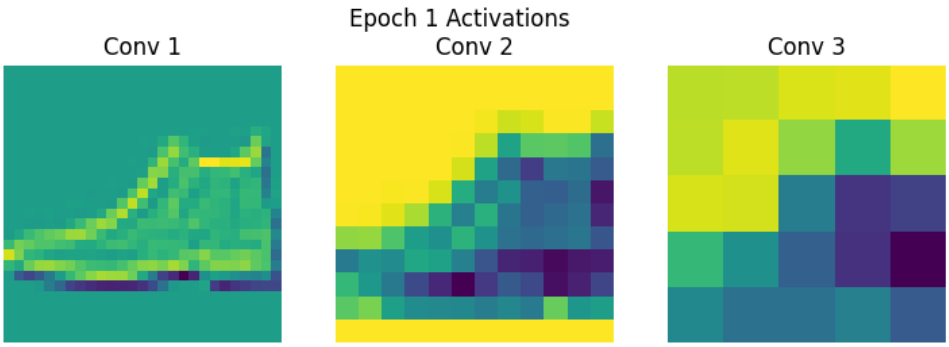

Figure 23: Epoch 1 Activations (Dropout Model)

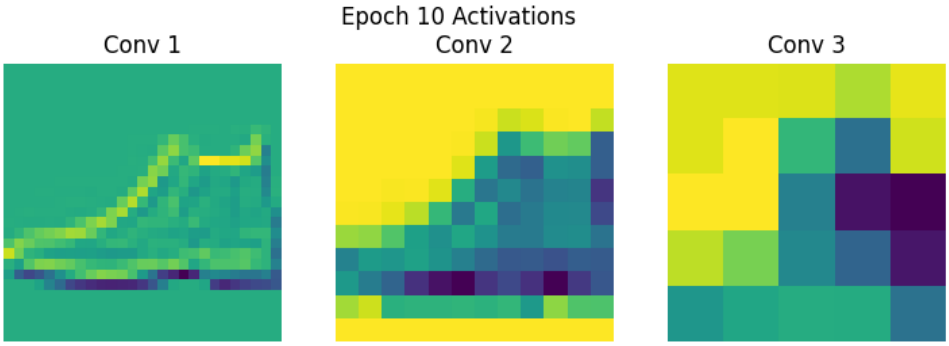

Figure 24: Epoch 10 Activations (Dropout Model)

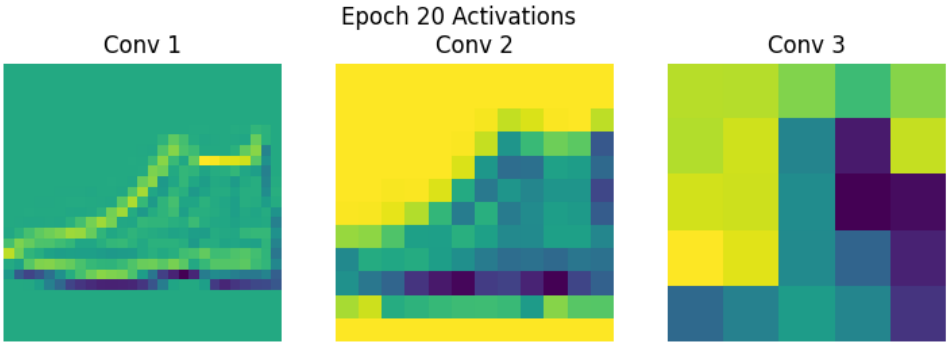

Figure 25: Epoch 20 Activations (Dropout Model)

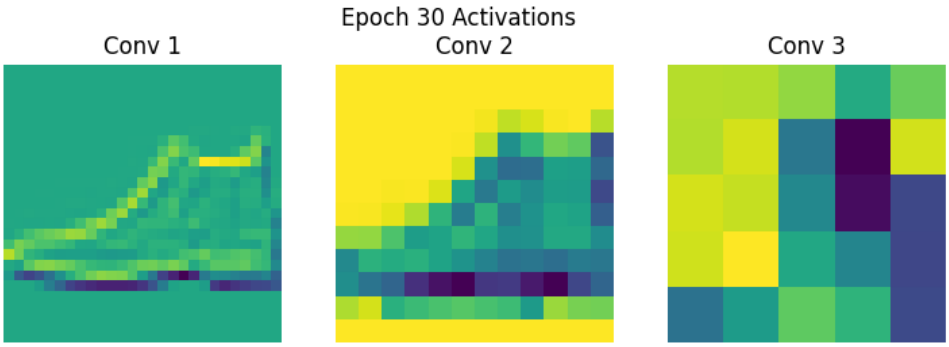

Figure 26: Epoch 30 Activations (Dropout Model)

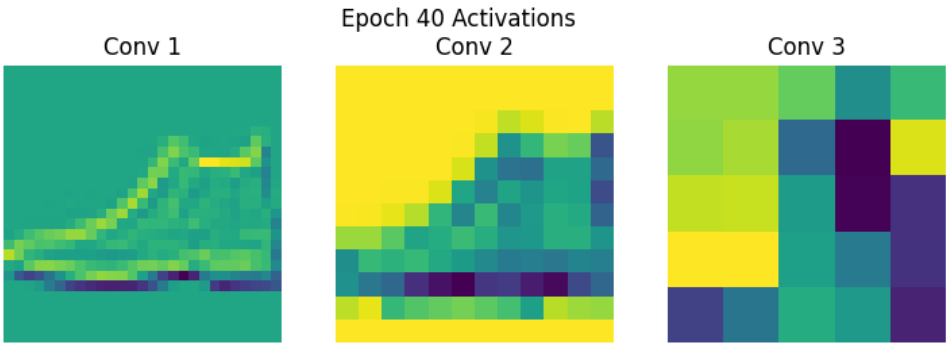

Figure 27: Epoch 40 Activations (Dropout Model)

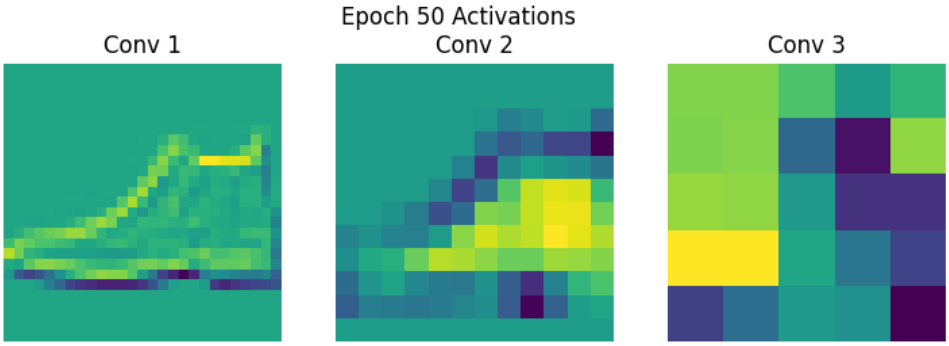

Figure 28: Epoch 50 Activations (Dropout Model)

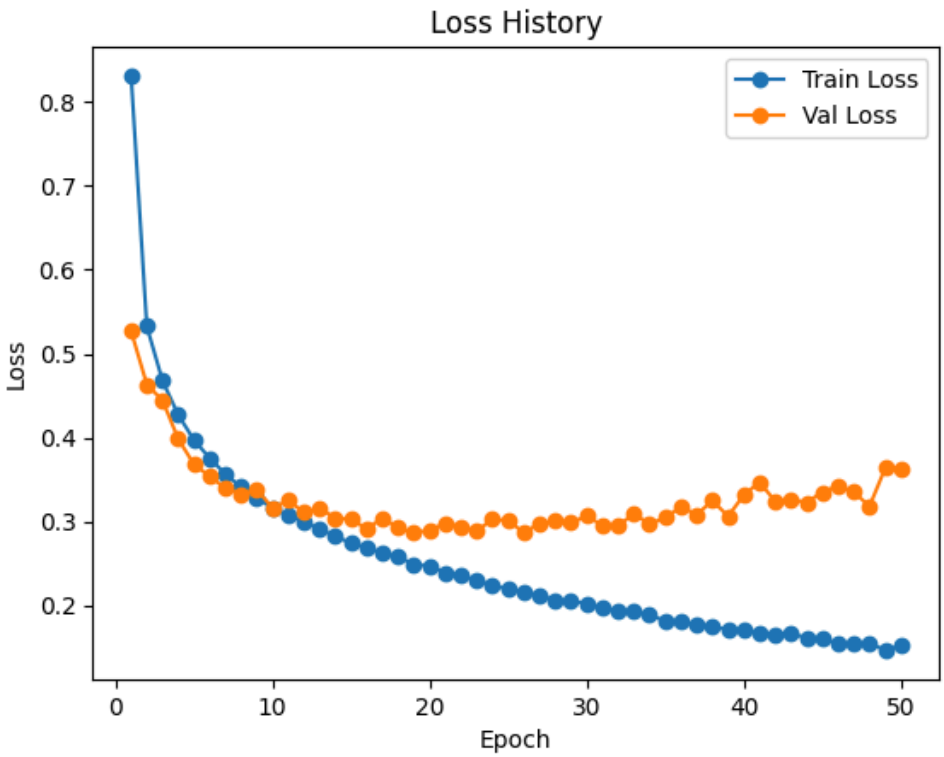

Figure 29: Loss History (Dropout Model)

Observations

The dropout model achieves a test accuracy of 89.19%, showing a slight improvement over the baseline. Compared to the models without dropout (baseline, KS/Stride, shallower), this model

exhibits a smaller gap between training and validation loss, indicating that dropout helps in regularizing the model and reducing overfitting. However, the overall accuracy is not as high as the KS/Stride or shallower models.

Experiment 5: Batch Normalisation

This experiment applies batch normalisation to the best-performing model identified from the previous experiments. The purpose is to assess if batch normalisation can further improve performance and stability.

Batch Normalisation Model Training

```python
results = {
    "Baseline": baseline_acc,
    "KS/Stride": ks_acc,
    "Shallow": shallow_acc,
    "Dropout": dropout_acc
}
best_name = max(results, key=results.get)
print(f"Best performing model so far: {best_name} ({results[best_name]:.2f}%)")

if best_name == "Baseline":
    model_best_bn = BaseCNN(k=2, s=1, num_blocks=3, filters=64, fc_hidden=64, dropout_p=0.0, batch_norm=True)
elif best_name == "KS/Stride":
    model_best_bn = BaseCNN(k=5, s=2, num_blocks=3, filters=64, fc_hidden=64, dropout_p=0.0, batch_norm=True)
elif best_name == "Shallow":
    model_best_bn = BaseCNN(k=2, s=1, num_blocks=2, filters=64, fc_hidden=64, dropout_p=0.0, batch_norm=True)
else:
    model_best_bn = BaseCNN(k=2, s=1, num_blocks=3, filters=64, fc_hidden=64, dropout_p=0.25, batch_norm=True)

print("Retraining with Batch Normalization...")
best_bn_acc = train_and_report(model_best_bn, num_epochs=50)
```

Result

```
Best performing model so far: Shallow (90.24%)
Retraining with Batch Normalization...
Total Trainable Parameters: 165194
Epoch 1/50, Train Loss: 0.4191, Val Loss: 0.3704
Epoch 10/50, Train Loss: 0.1354, Val Loss: 0.2513
Epoch 20/50, Train Loss: 0.0496, Val Loss: 0.3380
Epoch 30/50, Train Loss: 0.0197, Val Loss: 0.4337
Epoch 40/50, Train Loss: 0.0210, Val Loss: 0.5534
Epoch 50/50, Train Loss: 0.0204, Val Loss: 0.5564
Final Accuracies: Train: 99.87%, Val: 91.62%, Test: 91.13%
```

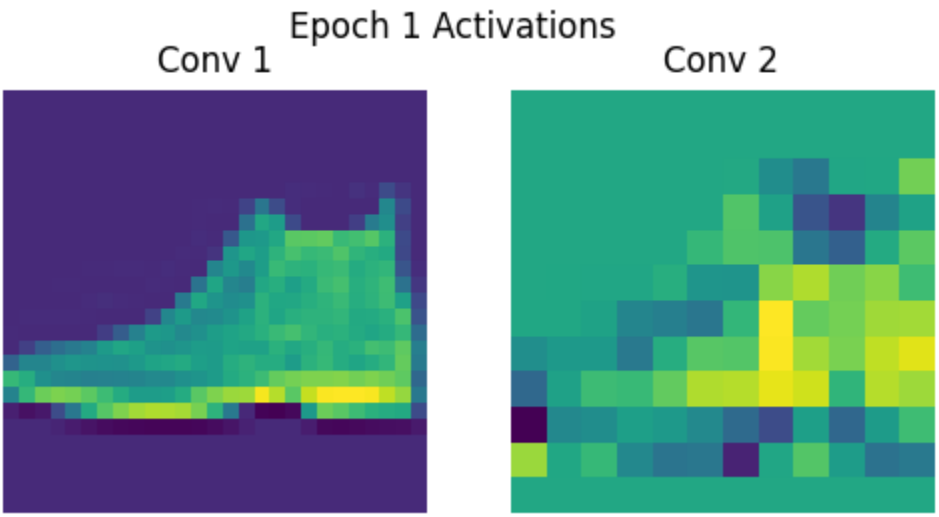

Figure 30: Epoch 1 Activations (Batch Normalisation Model)

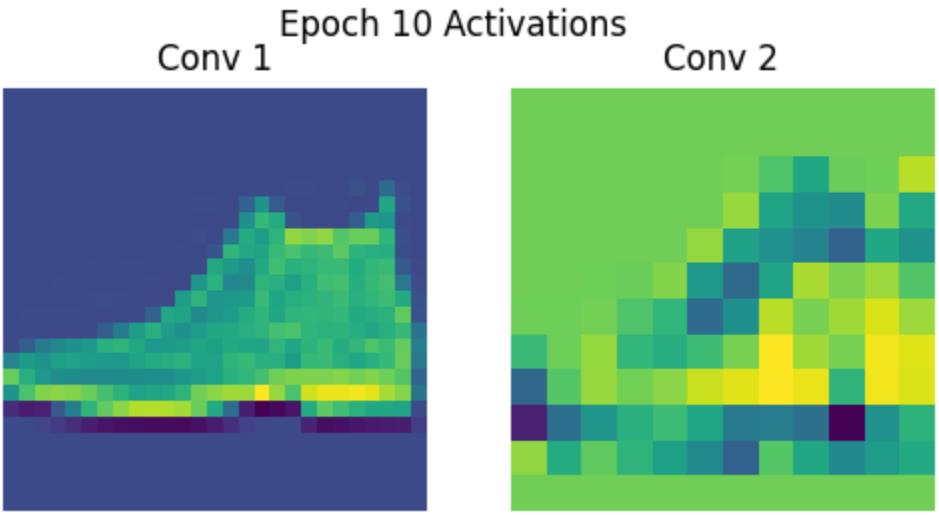

Figure 31: Epoch 10 Activations (Batch Normalisation Model)

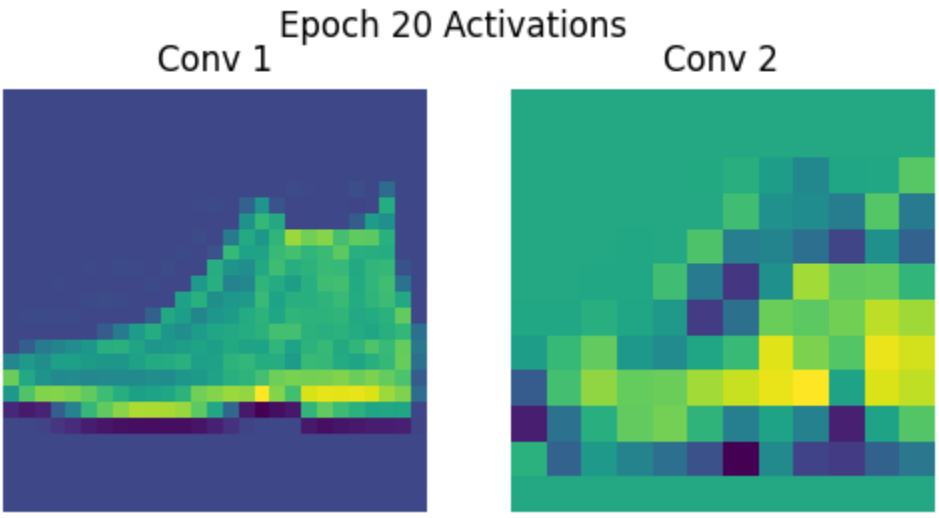

Figure 32: Epoch 20 Activations (Batch Normalisation Model)

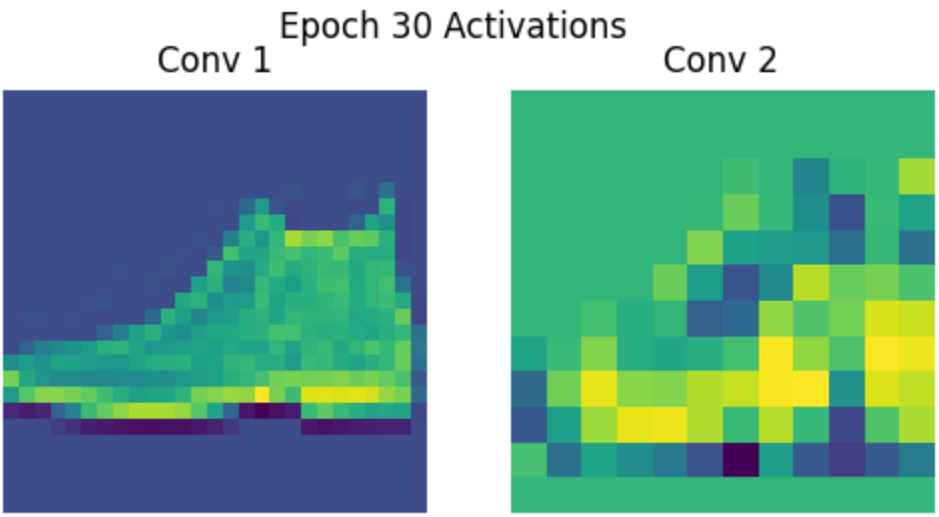

Figure 33: Epoch 30 Activations (Batch Normalisation Model)

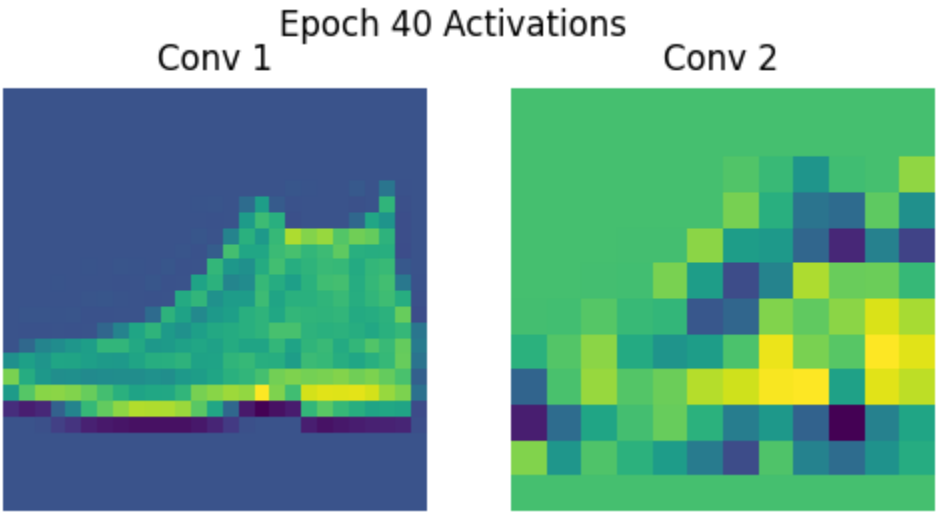

Figure 34: Epoch 40 Activations (Batch Normalisation Model)

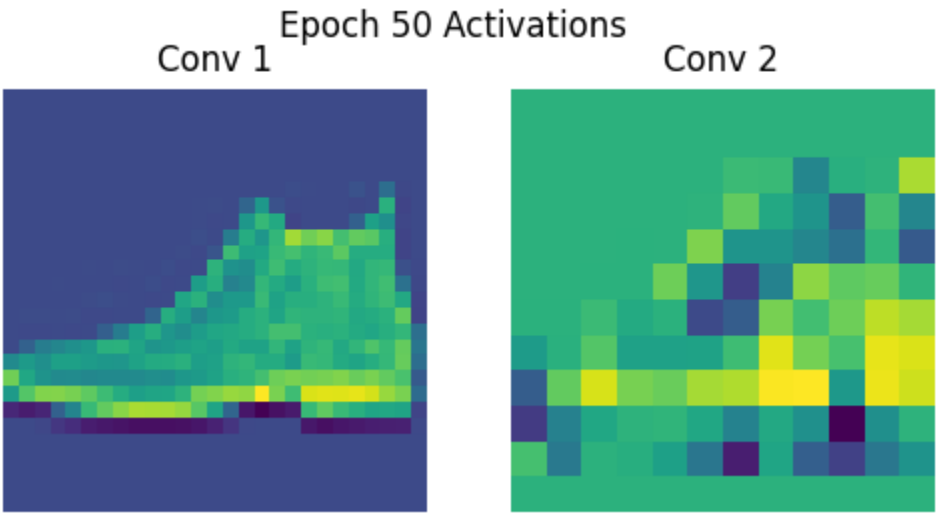

Figure 35: Epoch 50 Activations (Batch Normalisation Model)

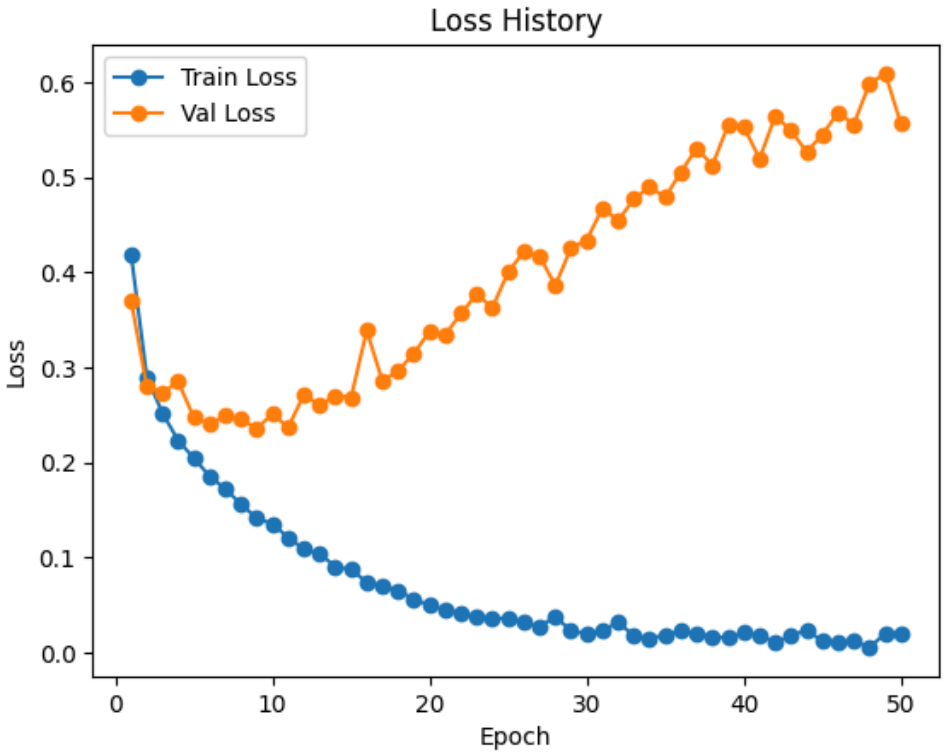

Figure 36: Loss History (Batch Normalisation Model)

Observations

The model with batch normalisation, based on the "Shallow" architecture, achieves the highest test accuracy of 91.13%. This indicates that batch normalisation significantly improves the model's performance and potentially its stability. While training loss is very low, there is still some gap with validation loss, but it's better managed compared to models without batch normalisation.

Conclusion

1. The baseline model achieved a test accuracy of 88.24%, demonstrating a reasonable starting point for classification.
2. Modifying kernel size and stride (Experiment 2) improved accuracy to 90.19%, but also led to increased overfitting as evidenced by the divergence of training and validation loss.
3. Reducing the network's depth (Experiment 3) yielded a comparable test accuracy of 90.24% but still exhibited overfitting.
4. Implementing dropout regularisation (Experiment 4) helped mitigate overfitting, as indicated by a smaller gap between training and validation loss, and achieved a test accuracy of 89.19%.
5. The most significant improvement was observed with batch normalisation (Experiment 5), which, when applied to the best-performing "Shallow" architecture, resulted in the highest test accuracy of 91.13%. This indicates that batch normalisation not only enhances performance but also contributes to better model stability.

In conclusion, for this task, a shallower network combined with batch normalisation proved to be the most effective configuration, achieving superior generalisation performance on the Fashion MNIST dataset. This highlights the importance of regularisation techniques like batch normalisation in building robust CNN models.